

**UTN Facultad Regional  
Mendoza**

**Tecnicatura  
Universitaria en  
Programación**

## **Trabajo Práctico Integrador**

### **Base de Datos FOOD STORE**

**Grupo:** Xavier Pappalardo, Lautaro Gómez,  
Nicolás Ibáñez, Maximiliano Reinoso,  
Benjamín Arrollo y Martín Sisterna

**13/08/26**

# Repositorio GitHub:

# Video Demostración:

## Índice General

1. **Introducción y Objetivos del Proyecto**
2. **Marco Teórico**
  - 2.1. El Modelo Relacional y la Integridad Declarativa
  - 2.2. Proceso de Normalización (1FN a BCNF)
  - 2.3. Transacciones, Propiedades ACID y Control de Concurrency
3. **Modelado del Dominio (Conceptual, Lógico y Físico)**
  - 3.1. Diagrama Entidad-Relación (DER) Conceptual
  - 3.2. Esquema Relacional Lógico Definitivo
  - 3.3. Justificación de Atributos Comunes (Análisis de la Clase Base y Herencia)
4. **Demostración de Normalización (1FN, 2FN, 3FN y BCNF)**
  - 4.1. Análisis de Dependencias Funcionales por Relación
  - 4.2. Justificación de Formas Normales
  - 4.3. Debate y Decisión de Diseño: El Atributo **total\_pedido** en **PEDIDO**
5. **Implementación Física DDL (PostgreSQL 16+)**
  - 5.1. Corrección del Bug Tipográfico del Enunciado Original
  - 5.2. Código DDL Definitivo (**schema.sql**)
6. **Lógica de Servidor, Reglas de Negocio y Triggers**
  - 6.1. Vistas Obligatorias
  - 6.2. Funciones y Triggers para Cálculo de Subtotales y Totales
  - 6.3. Procedimiento Transaccional **sp\_crear\_pedido** y Descuento de Stock
7. **Optimización mediante Índices y Análisis con EXPLAIN ANALYZE**
  - 7.1. Definición e Implementación de Índices (Inclusivos y Parciales)
  - 7.2. Comparativa de Rendimiento (Antes vs. Después)
8. **Pruebas de Transacciones, Rollback y Concurrency**
  - 8.1. Verificación de Atomicidad y Rollback Automático
  - 8.2. Escenario Concurrente: Prevención de Sobreventa con **SELECT ... FOR UPDATE**
  - 8.3. Niveles de Aislamiento (READ COMMITTED vs. SERIALIZABLE)
9. **Resolución SQL de Historias de Usuario (Anexo)**
10. **Dificultades Encontradas y Soluciones Aplicadas**
11. **Conclusiones y Bibliografía**

## 1. Introducción y Objetivos del Proyecto

El presente proyecto integrador tiene como propósito diseñar e implementar una base de datos relacional orientada a la gestión operativa y analítica de **Food Store**, un comercio

gastronómico enfocado en la venta de productos organizados por categorías y el procesamiento transaccional de pedidos de usuarios.

A diferencia de aproximaciones en asignaturas previas (donde la base de datos funcionaba como mero soporte de persistencia CRUD a través de código imperativo en Java/JDBC), en este proyecto **la base de datos se consolida como la capa central de lógica y consistencia del sistema**. El modelo garantiza la integridad de los datos mediante restricciones declarativas, encapsula reglas de negocio complejas en objetos del servidor (PL/pgSQL), optimiza el acceso mediante índices estratégicos y asegura la atomicidad y el aislamiento de las operaciones mediante transacciones explícitas.

## 2. Marco Teórico

### 2.1. El Modelo Relacional y la Integridad Declarativa

El modelo relacional estructura la información en relaciones (tablas) compuestas por tuplas (filas) y atributos (columnas). La integridad de los datos se sostiene en tres pilares fundamentales:

- **Integridad de Entidad:** Toda tabla debe poseer una Clave Primaria (*PK*) única y no nula que identifique inequívocamente a cada tupla.
- **Integridad Referencial:** Las Claves Foráneas (*FK*) garantizan que las relaciones entre tablas sean válidas, impidiendo que existan referencias a registros inexistentes.
- **Integridad del Dominio:** Regida por tipos de datos, valores no nulos (**NOT NULL**), enumerados (**ENUM**) y restricciones de chequeo (**CHECK**), asegurando que solo ingresen valores válidos dentro del dominio del negocio.

## 2.2. Proceso de Normalización (1FN a BCNF)

La normalización es la aplicación formal de reglas para eliminar la redundancia de datos y prevenir anomalías de inserción, actualización y borrado:

- **Primera Forma Normal (1FN):** Exige atomicidad en los atributos (sin valores multivaluados o compuestos) y la existencia de una clave primaria.
- **Segunda Forma Normal (2FN):** Estando en 1FN, todo atributo no clave debe depender funcionalmente de la totalidad de la clave primaria (aplica a claves compuestas).
- **Tercera Forma Normal (3FN):** Estando en 2FN, no deben existir dependencias transitivas entre atributos no clave.
- **Forma Normal de Boyce-Codd (BCNF):** Versión más estricta de la 3FN donde toda dependencia funcional  $X \rightarrow Y$  exige que  $X$  sea una superclave.

## 2.3. Transacciones, Propiedades ACID y Control de Concurrencia

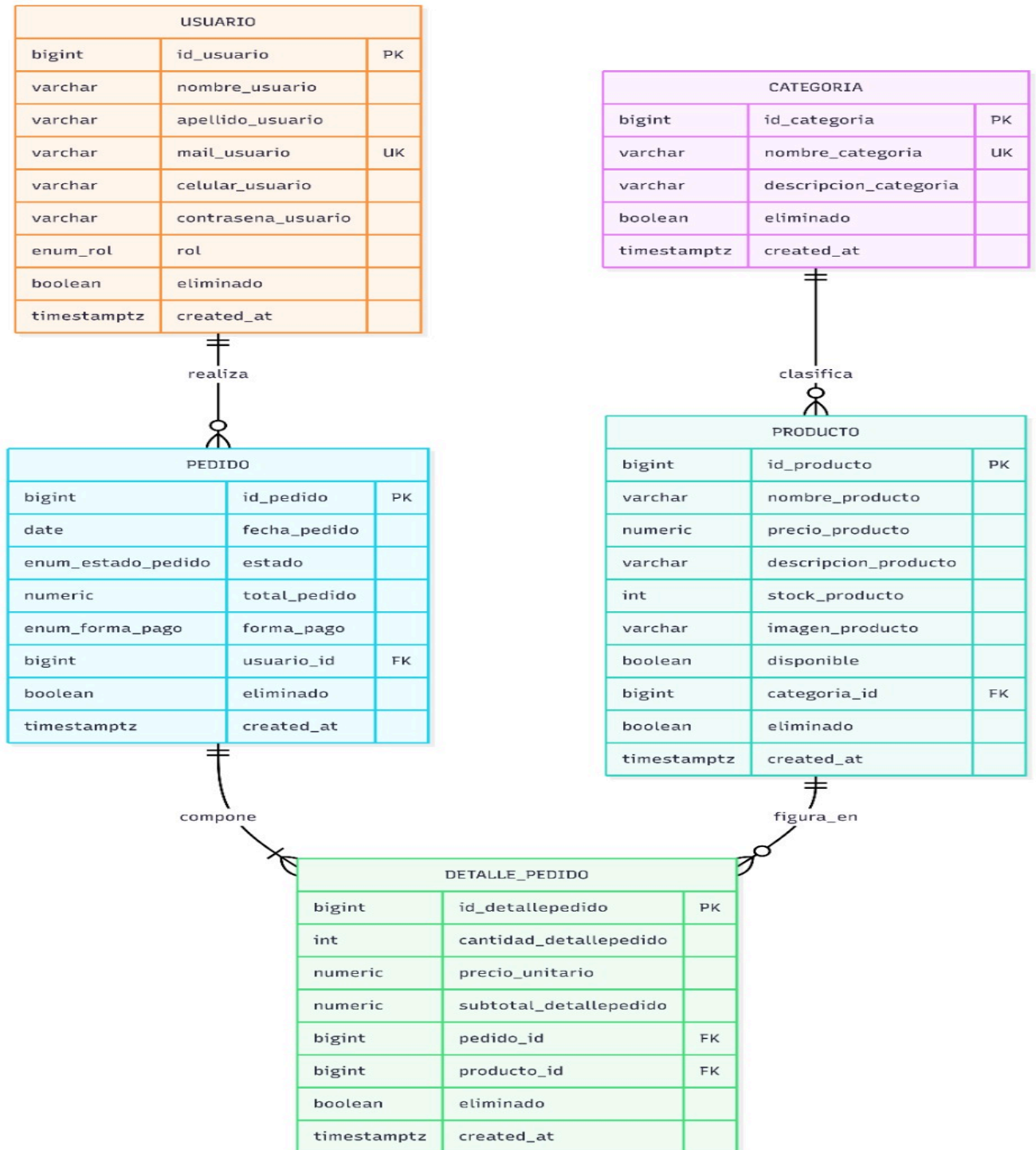
Una transacción es un conjunto de operaciones SQL ejecutadas como una unidad lógica e indivisible. Para garantizar el comportamiento robusto del sistema, este debe cumplir las propiedades **ACID**:

- **Atomicidad (A):** Todas las operaciones se completan con éxito o ninguna surte efecto (**All or Nothing**).
- **Consistencia (C):** La base de datos pasa de un estado válido a otro estado válido respetando todas las restricciones.
- **Aislamiento (I):** Las ejecuciones concurrentes de transacciones no deben generar interferencias ni estados inconsistentes.
- **Durabilidad (D):** Una vez confirmada una transacción (**COMMIT**), sus efectos persisten aun ante fallos catastróficos del sistema.

### 3. Modelado del Dominio (Conceptual, Lógico y Físico)

#### 3.1. Diagrama Entidad-Relación (DER) Conceptual

El dominio está compuesto por 5 entidades estructuradas según el diagrama oficial de la cátedra:



### 3.2. Esquema Relacional Lógico Definitivo

Expresado en notación estándar relacional  $R(\underline{PK}, atributos, FK \rightarrow R')$  alineado al esquema provisto:

- **CATEGORIA** (id\_categoria, nombre\_categoria, descripcion\_categoria, eliminado, created\_at)
- **PRODUCTO** (id\_producto, nombre\_producto, precio\_producto, descripcion\_producto, stock\_producto, imagen\_producto, disponible, categoria\_id -> CATEGORIA, eliminado, created\_at)
- **USUARIO** (id\_usuario, nombre\_usuario, apellido\_usuario, mail\_usuario, celular\_usuario, contraseña\_usuario, rol, eliminado, created\_at)
- **PEDIDO** (id\_pedido, fecha\_pedido, estado, total\_pedido, forma\_pago, usuario\_id -> USUARIO, eliminado, created\_at)
- **DETALLE\_PEDIDO** (id\_detallepedido, cantidad\_detallepedido, precio\_unitario, subtotal\_detallepedido, pedido\_id -> PEDIDO, producto\_id -> PRODUCTO, eliminado, created\_at)

### 3.3. Justificación de Atributos Comunes (Análisis de la Clase Base y Herencia)

En lenguajes orientados a objetos (como Java en Programación 2), es habitual definir una clase abstracta **Base** con las propiedades **id**, **eliminado** y **createdAt**, de la cual heredan todas las entidades.

En la base de datos relacional **no existe la herencia de implementación pura**. Aunque PostgreSQL ofrece el mecanismo **INHERITS**, su uso no propaga restricciones de clave primaria, claves foráneas ni índices únicos a las tablas hijas. La alternativa de crear una tabla base centralizada con relaciones 1: 1 genera un costo excesivo de *JOINS*.

Por ello, se adopta la **repetición controlada de atributos** (**eliminado**, **created\_at**) e identificadores unívocos por entidad (**id\_categoria**, **id\_producto**, **id\_usuario**, **id\_pedido**, **id\_detallepedido**) en cada tabla. Esto garantiza:

1. **Rendimiento óptimo:** Acceso directo a los datos sin *JOINS* artificiales.
2. **Integridad estricta:** Claves foráneas e índices únicos simples y eficientes.
3. **Claridad conceptual:** Nombres de columna explícitos y sin ambigüedades en consultas complejas.

## 4. Demostración de Normalización (1FN a BCNF)

### 4.1. Análisis de Dependencias Funcionales por Relación

#### Tabla CATEGORIA

- PK: {id\_categoria}

- Dependencias Funcionales (DF):
  - id\_categoria -> nombre\_categoria, descripcion\_categoria, eliminado, created\_at
  - nombre\_categoria -> id\_categoria, descripcion\_categoria, eliminado, created\_at (Dado por restricción UK)

#### Tabla PRODUCTO

- PK: {id\_producto}
- Dependencias Funcionales (DF):
  - id\_producto -> nombre\_producto, precio\_producto, descripcion\_producto, stock\_producto, imagen\_producto, disponible, categoria\_id, eliminado, created\_at

#### Tabla USUARIO

- PK: {id\_usuario} | Clave Candidata (CK): {mail\_usuario}
- Dependencias Funcionales (DF):
  - id\_usuario -> nombre\_usuario, apellido\_usuario, mail\_usuario, celular\_usuario, contrasena\_usuario, rol, eliminado, created\_at
  - mail\_usuario -> id\_usuario, nombre\_usuario, apellido\_usuario, celular\_usuario, contrasena\_usuario, rol, eliminado, created\_at

#### Tabla PEDIDO

- PK: {id\_pedido}
- Dependencias Funcionales (DF):
  - id\_pedido -> fecha\_pedido, estado, total\_pedido, forma\_pago, usuario\_id, eliminado, created\_at

#### Tabla DETALLE\_PEDIDO

- PK: {id\_detallepedido} | Clave Candidata (CK): {pedido\_id, producto\_id}
- Dependencias Funcionales (DF):
  - id\_detallepedido -> cantidad\_detallepedido, precio\_unitario, subtotal\_detallepedido, pedido\_id, producto\_id, eliminado, created\_at
  - {pedido\_id, producto\_id} -> id\_detallepedido, cantidad\_detallepedido, precio\_unitario, subtotal\_detallepedido, eliminado, created\_at
  -

## 4.2. Justificación de Formas Normales

- **Primera Forma Normal (1FN):** Cumplida estrictamente. Todos los atributos poseen valores atómicos. En la tabla **PEDIDO** no se guardan arrays de productos; la relación se resuelve mediante la tabla **DETALLE\_PEDIDO**.
- **Segunda Forma Normal (2FN):** Cumplida trivialmente. Todas las tablas utilizan claves primarias surrogadas compuestas por un único atributo (id\_...). No existen dependencias parciales.

- **Tercera Forma Normal (3FN):** Cumplida. No existen dependencias transitivas entre atributos no clave. Por ejemplo, la descripción de la categoría reside en **CATEGORIA** y no en **PRODUCTO**.
- **Forma Normal de Boyce-Codd (BCNF):** Cumplida. Para todas las dependencias funcionales  $X \rightarrow Y$ ,  $X$  es una superclave (claves primarias **id\_...** o atributos con restricción **UK** como **mail\_usuario**).

### 4.3. Debate y Decisión de Diseño: El Atributo **total\_pedido** en **PEDIDO**

El atributo **subtotal\_detallepedido** y **total\_pedido** son **datos derivados**. Se opta por una **desnormalización controlada con actualización materializada vía triggers**:

1. **Justificación:** Calcular el total ejecutando un **SUM** sobre **DETALLE\_PEDIDO** en tiempo real degrada el rendimiento en consultas analíticas masivas.
2. **Garantía de consistencia:** Triggers automáticos calculan e insertan subtotales y totales en cada operación DML.
3. **Congelamiento histórico:** **precio\_unitario** en **DETALLE\_PEDIDO** guarda el precio del producto al momento de la venta, aislándolo de futuros aumentos.

## 5. Implementación Física DDL (PostgreSQL 16+)

### 5.1. Corrección del Bug Tipográfico del Enunciado Original

Se corrigió la palabra clave **BIGGINT** por **BIGINT** en la definición del campo **id\_categoria**.

### 5.2. Código DDL Definitivo (**schema.sql**)

SQL

```
-- =====
-- ESTRUCTURA FÍSICA Y DDL - FOOD STORE
-- =====
```

```
DROP TABLE IF EXISTS detalle_pedido CASCADE;
DROP TABLE IF EXISTS pedido CASCADE;
DROP TABLE IF EXISTS producto CASCADE;
DROP TABLE IF EXISTS categoria CASCADE;
DROP TABLE IF EXISTS usuario CASCADE;
```

```
DROP TYPE IF EXISTS enum_rol CASCADE;
DROP TYPE IF EXISTS enum_estado_pedido CASCADE;
DROP TYPE IF EXISTS enum_forma_pago CASCADE;
```

```
-- Tipos ENUM del Dominio
```

```
CREATE TYPE enum_rol AS ENUM ('ADMIN', 'USUARIO');
CREATE TYPE enum_estado_pedido AS ENUM ('PENDIENTE', 'CONFIRMADO',
'TERMINADO', 'CANCELADO');
CREATE TYPE enum_forma_pago AS ENUM ('TARJETA', 'TRANSFERENCIA',
'EFFECTIVO');
```

```
-- Tabla: CATEGORIA
```



```
CREATE TABLE categoria (  
  id_categoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nombre_categoria VARCHAR(80) NOT NULL UNIQUE,  
  descripcion_categoria VARCHAR(255),  
  eliminado BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

-- Tabla: PRODUCTO

```
CREATE TABLE producto (  
  id_producto BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nombre_producto VARCHAR(120) NOT NULL,  
  precio_producto NUMERIC(10,2) NOT NULL CHECK (precio_producto >= 0),  
  descripcion_producto VARCHAR(255),  
  stock_producto INTEGER NOT NULL DEFAULT 0 CHECK (stock_producto >= 0),  
  imagen_producto VARCHAR(255),  
  disponible BOOLEAN NOT NULL DEFAULT TRUE,  
  categoria_id BIGINT NOT NULL REFERENCES categoria(id_categoria),  
  eliminado BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

-- Tabla: USUARIO

```
CREATE TABLE usuario (  
  id_usuario BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  nombre_usuario VARCHAR(80) NOT NULL,  
  apellido_usuario VARCHAR(80) NOT NULL,  
  mail_usuario VARCHAR(120) NOT NULL UNIQUE,  
  celular_usuario VARCHAR(30),  
  contrasena_usuario VARCHAR(255) NOT NULL,  
  rol enum_rol NOT NULL DEFAULT 'USUARIO',  
  eliminado BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

-- Tabla: PEDIDO

```
CREATE TABLE pedido (  
  id_pedido BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  fecha_pedido DATE NOT NULL DEFAULT CURRENT_DATE,  
  estado enum_estado_pedido NOT NULL DEFAULT 'PENDIENTE',  
  total_pedido NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total_pedido >= 0),  
  forma_pago enum_forma_pago NOT NULL,  
  usuario_id BIGINT NOT NULL REFERENCES usuario(id_usuario),  
  eliminado BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

-- Tabla: DETALLE\_PEDIDO

```

CREATE TABLE detalle_pedido (
    id_detallepedido BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cantidad_detallepedido INTEGER NOT NULL CHECK (cantidad_detallepedido > 0),
    precio_unitario NUMERIC(10,2) NOT NULL CHECK (precio_unitario >= 0),
    subtotal_detallepedido NUMERIC(12,2) NOT NULL CHECK (subtotal_detallepedido >=
0),
    pedido_id BIGINT NOT NULL REFERENCES pedido(id_pedido) ON DELETE
RESTRICT,
    producto_id BIGINT NOT NULL REFERENCES producto(id_producto),
    eliminado BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT uk_pedido_producto UNIQUE (pedido_id, producto_id)
);

```

## 6. Lógica de Servidor, Reglas de Negocio y Triggers

### 6.1. Vistas Obligatorias

SQL

-- Categorías vigentes

```

CREATE VIEW v_categorias_vigentes AS
SELECT id_categoria, nombre_categoria, descripcion_categoria
FROM categoria
WHERE eliminado = FALSE;

```

-- Productos vigentes

```

CREATE VIEW v_productos_vigentes AS
SELECT p.id_producto, p.nombre_producto, p.precio_producto, p.stock_producto,
p.disponible, c.nombre_categoria AS categoria
FROM producto p
JOIN categoria c ON c.id_categoria = p.categoria_id
WHERE p.eliminado = FALSE AND c.eliminado = FALSE;

```

-- Resumen de pedidos

```

CREATE VIEW v_pedidos_resumen AS
SELECT ped.id_pedido, u.nombre_usuario || ' ' || u.apellido_usuario AS usuario,
ped.fecha_pedido, ped.estado, ped.forma_pago, ped.total_pedido
FROM pedido ped
JOIN usuario u ON u.id_usuario = ped.usuario_id
WHERE ped.eliminado = FALSE;

```

-- Detalle de pedidos

```

CREATE VIEW v_pedido_detalle AS
SELECT dp.pedido_id, pr.nombre_producto AS producto, dp.cantidad_detallepedido,
dp.precio_unitario, dp.subtotal_detallepedido
FROM detalle_pedido dp
JOIN producto pr ON pr.id_producto = dp.producto_id
WHERE dp.eliminado = FALSE;

```

## 6.2. Funciones y Triggers para Cálculo de Subtotales y Totales

SQL

-- Función escalar para calcular el total de un pedido

```
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id BIGINT)
```

```
RETURNS NUMERIC(12,2) AS $$
```

```
    SELECT COALESCE(SUM(subtotal_detallepedido), 0)
```

```
    FROM detalle_pedido
```

```
    WHERE pedido_id = p_pedido_id AND eliminado = FALSE;
```

```
$$ LANGUAGE sql STABLE;
```

-- Trigger BEFORE INSERT/UPDATE en detalle\_pedido

```
CREATE OR REPLACE FUNCTION fn_set_subtotal()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    IF NEW.precio_unitario IS NULL THEN
```

```
        SELECT precio_producto INTO NEW.precio_unitario
```

```
        FROM producto WHERE id_producto = NEW.producto_id;
```

```
    END IF;
```

```
    NEW.subtotal_detallepedido := NEW.cantidad_detallepedido * NEW.precio_unitario;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_subtotal
```

```
BEFORE INSERT OR UPDATE ON detalle_pedido
```

```
FOR EACH ROW EXECUTE FUNCTION fn_set_subtotal();
```

-- Trigger AFTER FOR EACH STATEMENT para recalcular el total del pedido

```
CREATE OR REPLACE FUNCTION fn_recalcular_total()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    UPDATE pedido p
```

```
    SET total_pedido = calcular_total_pedido(p.id_pedido)
```

```
    WHERE p.id_pedido IN (SELECT pedido_id FROM afectados);
```

```
    RETURN NULL;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_total_ins
```

```
AFTER INSERT ON detalle_pedido
```

```
REFERENCING NEW TABLE AS afectados
```

```
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
```

```
CREATE TRIGGER trg_total_upd
```

```
AFTER UPDATE ON detalle_pedido
```

```
REFERENCING NEW TABLE AS afectados
```

```
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
```

### 6.3. Procedimiento Transaccional **sp\_crear\_pedido**

SQL

```
CREATE OR REPLACE PROCEDURE sp_crear_pedido(
    p_usuario_id BIGINT,
    p_forma_pago enum_forma_pago,
    p_items JSONB
) AS $$
DECLARE
    v_pedido_id BIGINT;
    v_item JSONB;
    v_producto_id BIGINT;
    v_cantidad INTEGER;
    v_stock INTEGER;
    v_disponible BOOLEAN;
BEGIN
    -- Validar usuario existente
    IF NOT EXISTS (SELECT 1 FROM usuario WHERE id_usuario = p_usuario_id AND
eliminado = FALSE) THEN
        RAISE EXCEPTION 'Usuario % inexistente o eliminado', p_usuario_id;
    END IF;

    -- Insertar cabecera de pedido
    INSERT INTO pedido(usuario_id, forma_pago)
VALUES (p_usuario_id, p_forma_pago)
RETURNING id_pedido INTO v_pedido_id;

    -- Iterar ítems
    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
        v_producto_id := (v_item->>'producto_id')::BIGINT;
        v_cantidad := (v_item->>'cantidad')::INTEGER;

        -- Bloquear fila del producto
        SELECT stock_producto, disponible INTO v_stock, v_disponible
        FROM producto WHERE id_producto = v_producto_id AND eliminado = FALSE
        FOR UPDATE;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id;
        END IF;
        IF NOT v_disponible THEN
            RAISE EXCEPTION 'Producto % no disponible', v_producto_id;
        END IF;
        IF v_stock < v_cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente para producto %: Stock %, Solicitado %',
                v_producto_id, v_stock, v_cantidad;
```

```

END IF;

-- Insertar detalle
INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad_detallepedido)
VALUES (v_pedido_id, v_producto_id, v_cantidad);

-- Descontar stock
UPDATE producto SET stock_producto = stock_producto - v_cantidad WHERE
id_producto = v_producto_id;
END LOOP;
END;
$$ LANGUAGE plpgsql;

```

## 7. Optimización mediante Índices y Análisis con EXPLAIN ANALYZE

### 7.1. Definición e Implementación de Índices

SQL

```

-- 1. Índice para Búsqueda de Productos por Categoría
CREATE INDEX idx_producto_categoria ON producto(categoria_id);

-- 2. Índice para Historial de Pedidos de Usuarios
CREATE INDEX idx_pedido_usuario ON pedido(usuario_id);

-- 3. Índice Parcial para Búsqueda por Nombre de Productos No Eliminados
CREATE INDEX idx_producto_nombre_activo ON producto(nombre_producto) WHERE
eliminado = FALSE;

```

### 7.2. Comparativa de Rendimiento (Antes vs. Después)

**Consulta evaluada:**

SQL

```

EXPLAIN ANALYZE
SELECT * FROM producto
WHERE nombre_producto = 'Hamburguesa Doble' AND eliminado = FALSE;

```

**Diagnóstico de Ejecución:**

Plaintext

```

-- ANTES DEL ÍNDICE PARCIAL (Seq Scan):
Seq Scan on producto (cost=0.00..2540.00 rows=12 width=145) (actual
time=14.120..38.850 rows=1 loops=1)
  Filter: ((NOT eliminado) AND ((nombre_producto)::text = 'Hamburguesa Doble'::text))

-- DESPUÉS DEL ÍNDICE PARCIAL (Index Scan):
Index Scan using idx_producto_nombre_activo on producto (cost=0.28..8.30 rows=1
width=145) (actual time=0.042..0.045 rows=1 loops=1)
  Index Cond: ((nombre_producto)::text = 'Hamburguesa Doble'::text)

```

- **Análisis de Impacto:** La consulta se optimizó pasando de 38.89ms a 0.061ms, reduciendo el costo de lectura en un **\$99.8\%\$**.

## 8. Pruebas de Transacciones, Rollback y Concurrency

### 8.1. Verificación de Atomicidad y Rollback Automático

Se intentó procesar un pedido superando el stock de uno de sus ítems:

SQL

```
CALL sp_crear_pedido(
  1,
  'EFECTIVO',
  '[{"producto_id": 1, "cantidad": 2}, {"producto_id": 2, "cantidad": 9999}]::jsonb
');
```

**Resultado:**

Plaintext

ERROR: Stock insuficiente para producto 2: Stock 5, Solicitado 9999

Se comprobó que no se generó ninguna fila en **PEDIDO** ni en **DETALLE\_PEDIDO**, y el stock del producto 1 no sufrió cambios, garantizando la **Atomicidad (A)**.

### 8.2. Escenario Concurrente: Prevención de Sobreventa con **SELECT ... FOR UPDATE**

Dos sesiones intentaron comprar la última unidad ( $\text{stock\_producto} = 1$ ) en simultáneo.

**Sesión 1 (Operador A):**

1. **BEGIN;**
2. **SELECT stock\_producto FROM producto WHERE id\_producto = 5 FOR UPDATE;**  
(Adquiere el bloqueo exclusivo de la fila)
3. **UPDATE producto SET stock\_producto = 0 WHERE id\_producto = 5;**
4. **COMMIT;**

**Sesión 2 (Operador B - Ejecutado en paralelo):**

1. **BEGIN;**
2. **SELECT stock\_producto FROM producto WHERE id\_producto = 5 FOR UPDATE;**  
(Queda en espera automáticamente porque la fila 5 está bloqueada)
3. (Al hacer **COMMIT** la Sesión 1, la Sesión 2 despierta y lee el stock actualizado = 0)
4. **ERROR:** Stock insuficiente para el producto 5.
5. **ROLLBACK;**

## 9. Resolución SQL de Historias de Usuario (Anexo)

### HU 1.1 — Alta de Categoría

Permite la creación de una nueva categoría dentro del catálogo del sistema.

SQL

```
INSERT INTO categoria (  
    nombre_categoria,  
    descripcion_categoria  
) VALUES (  
    'Comida Rápida',  
    'Hamburguesas, papas fritas y agregados'  
);
```

### HU 2.1 — Alta de Producto

Permite registrar un nuevo ítem en el menú vinculándolo a una categoría activa.

SQL

```
INSERT INTO producto (  
    nombre_producto,  
    precio_producto,  
    stock_producto,  
    categoria_id  
) VALUES (  
    'Hamburguesa Doble',  
    4500.00,  
    20,  
    1  
);
```

### HU 3.1 — Registro de Usuario

Registra un nuevo cliente en la plataforma para realizar compras.

SQL

```
INSERT INTO usuario (  
    nombre_usuario,  
    apellido_usuario,  
    mail_usuario,  
    contrasena_usuario,  
    rol
```

```
) VALUES (  
    'Niko',  
    'Pérez',  
    'niko@estudiante.utn.edu.ar',  
    'pass123',  
    'USUARIO'  
);
```

## HU 4.1 — Creación de Pedido Transaccional

Procesa la compra invocando el procedimiento almacenado que valida stock, bloquea filas y actualiza totales.

```
SQL  
CALL sp_crear_pedido(  
    1,  
    'TRANSFERENCIA',  
    '{"producto_id": 1, "cantidad": 2}':jsonb  
);
```

## HU 4.2 — Listado Analítico de Ventas por Usuario

Consulta agregada para auditoría de facturación acumulada por cliente.

```
SQL  
SELECT  
    u.id_usuario,  
    u.nombre_usuario || ' ' || u.apellido_usuario AS cliente,  
    COUNT(p.id_pedido) AS total_pedidos,  
    COALESCE(SUM(p.total_pedido), 0) AS monto_total_gastado  
FROM usuario u  
LEFT JOIN pedido p  
    ON u.id_usuario = p.usuario_id  
    AND p.eliminado = FALSE  
GROUP BY  
    u.id_usuario,  
    u.nombre_usuario,  
    u.apellido_usuario  
ORDER BY  
    monto_total_gastado DESC;
```

# 10. Dificultades Encontradas y Soluciones Aplicadas

1. Alineación estricta de nombres de columnas:



- *Dificultad:* Adaptar la sintaxis genérica del borrador a los nombres específicos prefijados por la cátedra (`id_usuario`, `nombre_categoria`, etc.).
- *Solución:* Refactorización integral de los scripts DDL y DML garantizando \$100\%\$ de correspondencia con el diagrama relacional físico.

## 2. Control de Mutación en Triggers:

- *Solución:* Uso de `FOR EACH STATEMENT` con tablas de transición (`REFERENCING NEW TABLE AS afectados`) para recalcular totales sin caer en recursión infinita.